VIETNAM ACADEMY OF SCIENCE AND TECHNOLOGY
UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

BASIC PROGRAMMING

Lecture 1: Introduction to Programming with C

Dr. NGUYEN Hoang Ha
Dr. NGUYEN Minh Huong
Dr. LE Huu Ton

Course Information

- Prerequisites: Introduction to Informatics
- Materials:
  - Slides (uploaded on Moodle)
  - Suggested textbook: C Programming Language (2nd Edition) by Brain W. Kernighan and Dennis M. Ritchie
- Class organizing:
  - Attendance check: fingerprint, exercise submission…
- Assessment:
  - Practical exam (midterm-test): MCQ test
  - Theoretical exam (final-test): **live coding test**

Course Objectives

- Concept of Programming:

  How is a problem solved with computer?

  Problem → Algorithm → Computer Program
- To understand development process
  - Write source code
  - Compile/interpret
  - Tools: compiler/interpreter, editor
- To understand procedural programming essentials
  - Data types, variables, control structure…
- To obtain programming skills in C

Lecture 1: Introduction to Programming with C

- What is Computer Programming
- Overview of C
- Compiling a C Program
- Practical Issues
- Basic Coding Conventions in C

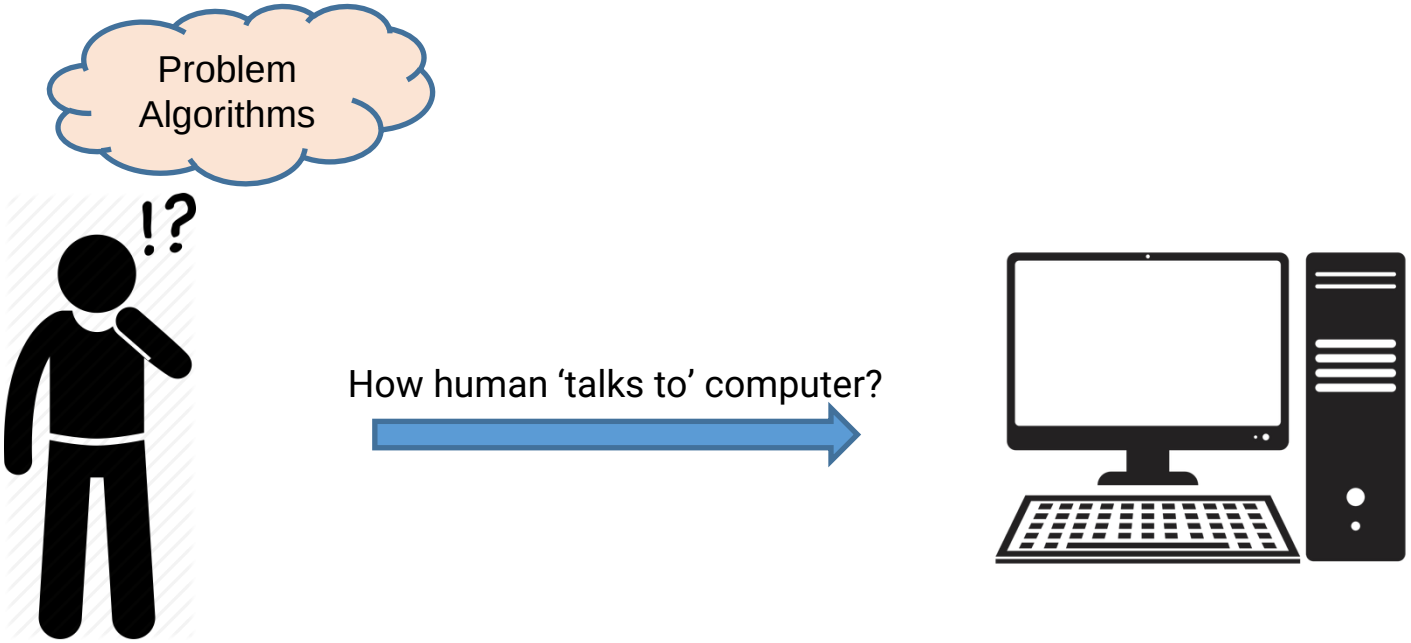
What is Computer Programming?



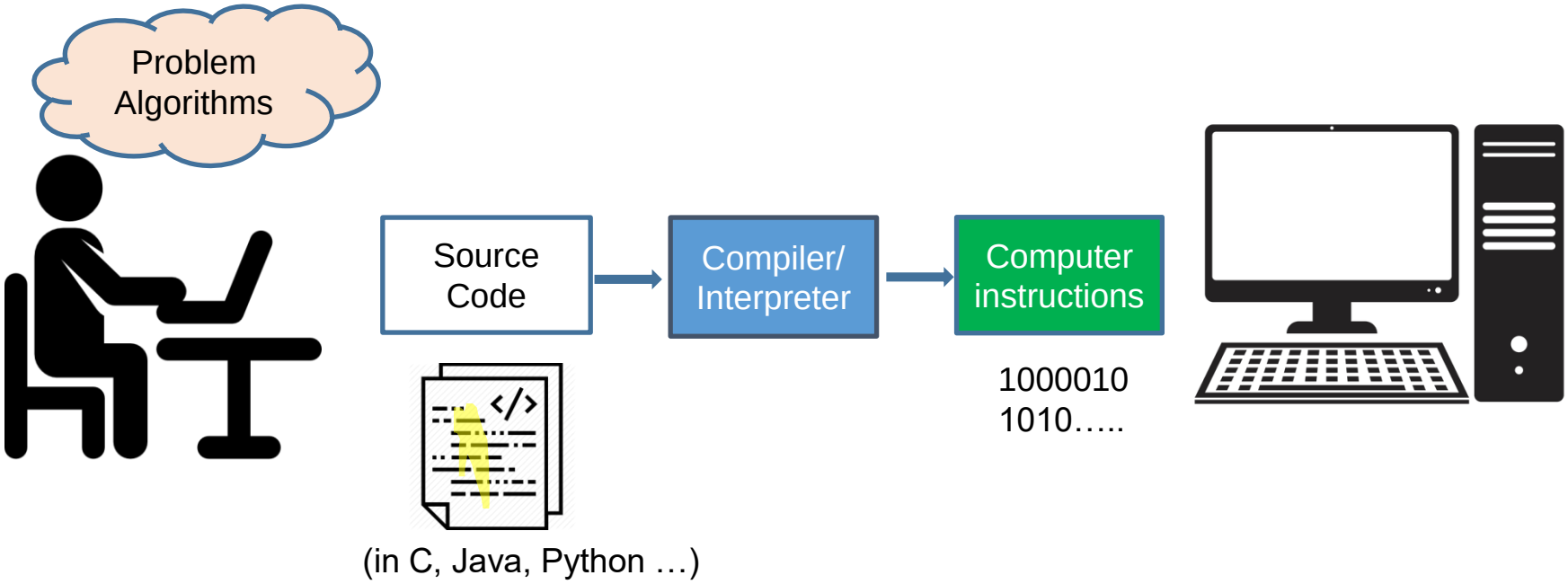
"Talk is cheap, show me the code"

Linus Torvalds, 2000

What is Computer Programming?

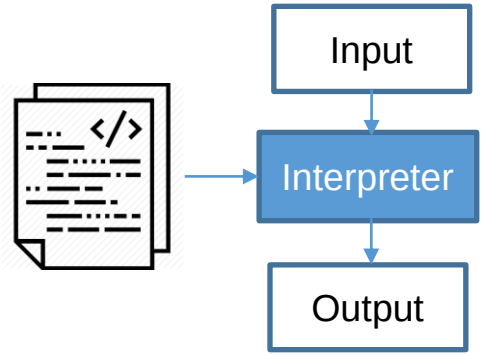


What is Computer Programming?



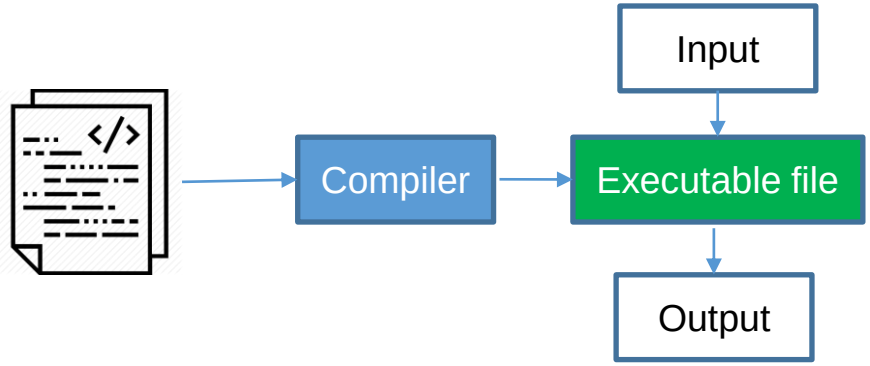
Interpretation vs. Compilation

Interpretation



- Machine code produced at runtimes
- More flexibility to change

Compilation



- Machine code produced once at compilation time
- Better performance

Computer programming Languages

A formal language comprising a set of instructions for computer that produce various kinds of output.



A simple Python Program

- Source code

```
# Python3 program to add two numbers
number1 = input("First number: ")
number2 = input("\nSecond number: ")

# Adding two numbers
# User might also enter float numbers
sum = float(number1) + float(number2)
 # Display the sum
# will print value in float
print("The sum of {0} and {1} is {2}" .format(number1, number2, sum))
```

- Running

```
First number: 13.5 Second number: 1.54
The sum of 13.5 and 1.54 is 15.04
```

Why C?

Top popular programming languages

TOP 10

Popular Programming Languages in 2020

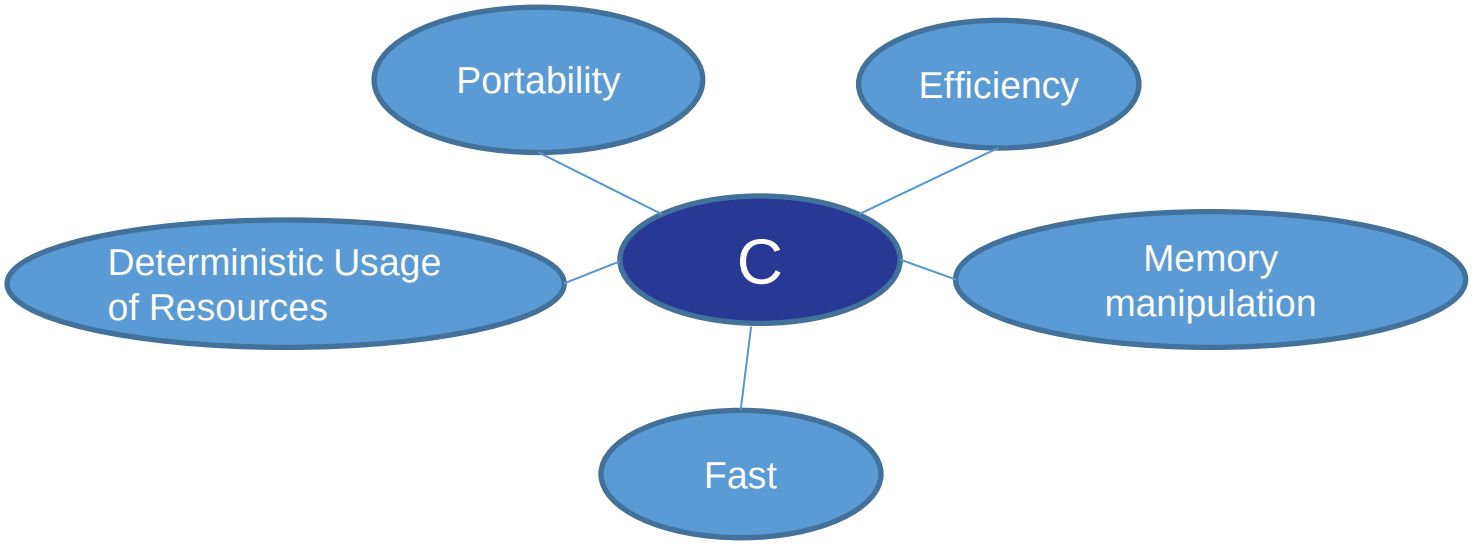
1	Python
2	JavaScript
3	Java
4	C#
5	C
6	C++
7	GO
8	R
9	Swift
10	PHP

WWW.NORTHEASTERN.EDU/GRADUATE



Why C?

- Concept for other programming languages
- Features of C:



- Uses today: operating systems, device driver, compiler or interpreter, embedded systems, databases …

Overview of C

History of C

- 1967: BCPL (by Martin Richards): Grandparent of C
  - Used for writing OS & compilers
- 1970: B (by Ken Thompson): Parent of C
- 1972: Traditional C evolved from B (by Denis Ritchie)
- 1978: The C Programming Language – language reference by Kernighan & Ritchie
- 1989: ANSI C (by ANSI Committee)
- 1990: ANSI/ISO C (by ISO Committee)
- 1999: C99 (by Standardization Committee)
- Parent of Objective C, C#, Objective C

Components of a C program

File hello.c

```c
#include <stdio.h> // preprocessor directives
int main() // Main function called by OS when user runs the program
{
        /* print out out a string */
        printf("Hello World"); //function call
        return 0; //give to OS exist value, 0 means successful
}
```

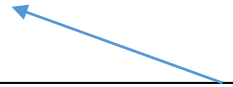
C Keywords and Identifiers

- Character Set:
  - Alphabets: uppercase and lower case
  - Digits: 0-9
  - Special characters
- Keywords:
  - Reserved words
  - fixed meaning, cannot be changed
  - Lowercase
  - Example: int, include, return…

C keywords and Identifiers (cont)

- Identifiers:
  - Names given to C entities (variable, functions, structures…)
  - Unique during execution
  - Rules for writing identifier:
    - Composed of letters, digits and underscore "_"
    - First letter: not a digit
  - Example:

```
int x;
```



Identifier 'x' is a name given to variable x

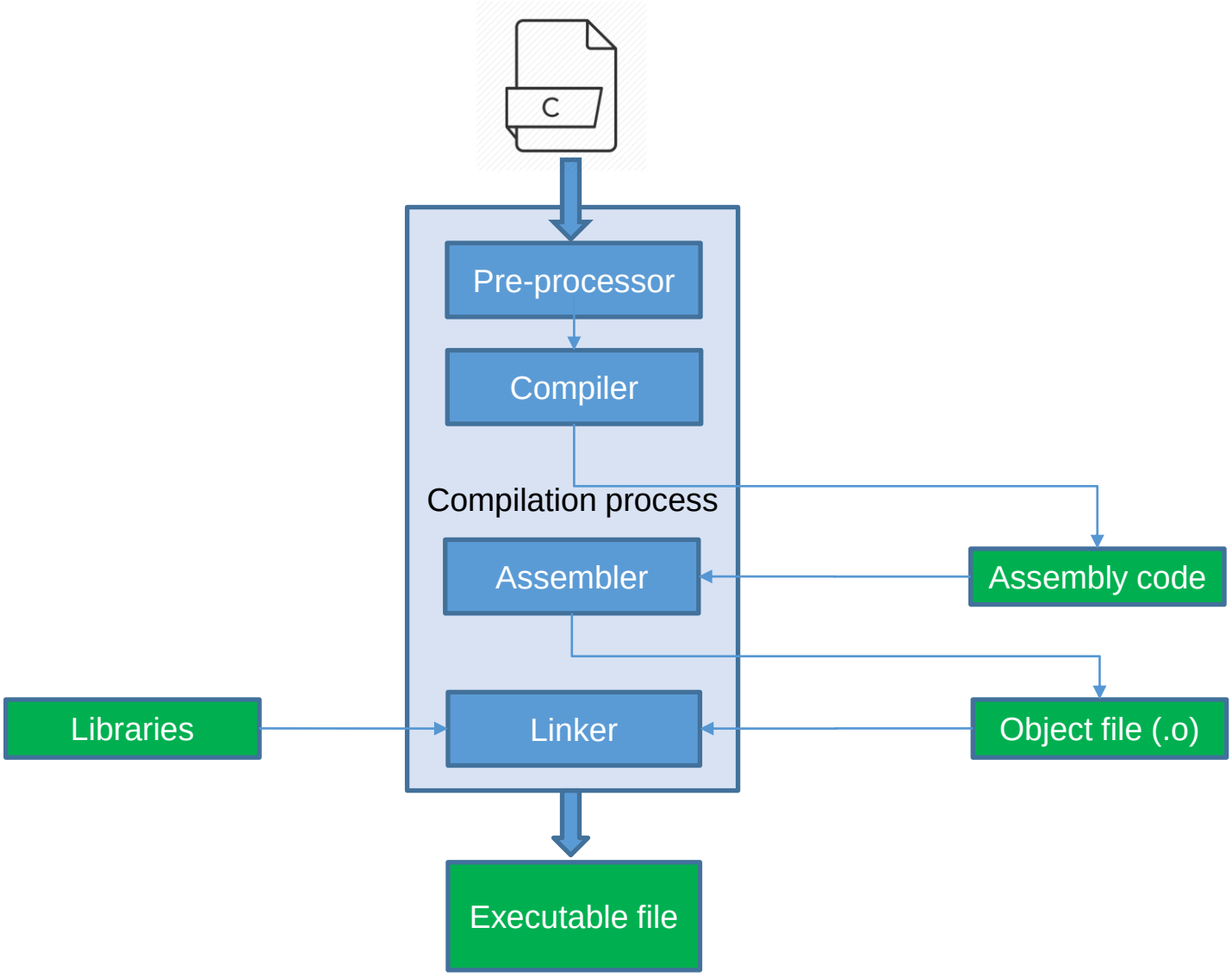
C Preprocessors

- Preprocessors: A text substitution instructing the compiler to do pre-processing
- Beginning with "#"
- Should be in the first column
- Examples:

```
#include <stdio.h>
#include "sayhello.h“
#define MAX_ARRAY_LENGTH 20
```

Compiling a C Program

Complication Process



The Preprocessor

- Getting rid of all the comments
- Including the code of header files
- Replacing all of the macros by their values
- Output: .i file
- How to stop after this step: Type: gcc –E hello.c

```
# 1 "hello.c"
# 1 "<built-in>" 1
# 1 "<built-in>" 3
# 361 "<built-in>" 3
# 1 "<command line>" 1
# 1 "<built-in>" 2
# 1 "hello.c" 2
# 1 "/usr/include/stdio.h" 1 3 4
# 64 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/_stdio.h" 1 3 4
# 68 "/usr/include/_stdio.h" 3 4
# 1 "/usr/include/sys/cdefs.h" 1 3 4
```

The Compiler

- Generating Intermediate Representation (IR): .s file
- How to stop after this step:

  Type: gcc –S hello.c

  Output file: hello.s

```
        .section        __TEXT,__text,regular,pure_instructions
        .macosx_version_min 10, 13
        .globl  _main                   ## -- Begin function main
        .p2align        4, 0x90
_main:                                  ## @main
        .cfi_startproc
## %bb.0:
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset %rbp, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register %rbp
        subq    $16, %rsp
```

The Assembler

- Transforming .s file into object code: .o file (in binary)
- How to stop after this step:

Type: gcc –c hello.c

Output file: hello.o

```
???? ?? ?__text__TEXT* ??__cstring__TEXT*
                                        J__compact_unwind__LD8 X?__eh_frame__TEXTX@x
                                                                                    h$
??
  PUH??H??H?=E???1ɟ E???H??]?Hello World*zRx
-                                        ?$????????*A?C
```

The tool gcc

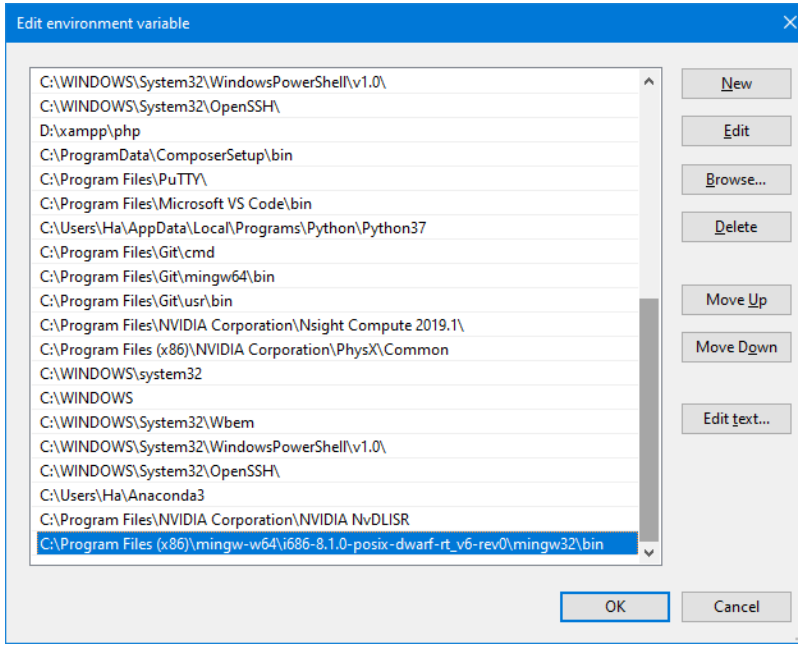
- What is gcc (GNU Compiler Collection)?
  - A compiler produced by GNU Project
  - Supporting various programming languages
- Our official C Compiler in this course



Practical Issues

Setup Development Environment on Windows

- Compiler & library
  - Install mingw package from http://mingw-w64.org
  - Add the path of binary of mingw to "PATH" variable environment



- Editor/IDE
  - Install Visual Studio Code
  - Add C/C++ plugin
  - Configure shortcut to compile (optional): Menu Terminal → Configure Default Build Task → select C/C++: gcc.exe build active file

Setup Development Environment on Linux

- Compiler & library
  - Check whether gcc is ready: *$gcc --version*
  - If not, then install: *$sudo apt install build-essential*
- Editor/IDE
  - Install Visual Studio Code
  - Add C/C++ plugin
  - Configure shortcut to compile (optional): Menu Terminal → Configure Default Build Task → select C/C++: gcc build active file

Setup Development Environment on MacOS

- Compiler & library
  - Check whether gcc is ready: *$gcc --version*
  - If not, then install: *$brew install gcc*
- Editor/IDE
  - Install Visual Studio Code
  - Add C/C++ plugin
  - Configure shortcut to compile (optional): Menu Terminal → Configure Default Build Task → select C/C++: gcc build active file

Start Writing a C Program

- Input
  - Feed some data into a program
  - From a file or from command line
- Output
  - Display some data on screen, printer or file
- Input and output functions:
  - getchar() and putchar()
  - scanf() and printf(): read and print a string (%s), integer(%d), character(%c) or float(%f)

→ Task: create a C program to print out "Hello world" in the screen

Compiling with GCC

- Step 1: Check gcc availability and install (if not available)

  gcc -v
- Step 2: Create a C program in an editor
  - Suggest: Visual Studio Code
  - hello.c

Compiling with GCC (cont)

- Step 3: Open terminal, go to the directory of the C file

```
Mac-mini-2:source code minhhuongnguyen$ ls
hello.c          sayhello.c       sayhello.h
```

- Step 4: Compiling

  gcc hello.c

```
Mac-mini-2:source code minhhuongnguyen$ gcc hello.c
Mac-mini-2:source code minhhuongnguyen$ ls
a.out            hello.c          sayhello.c       sayhello.h
```

Run the Compiled Program

- Executing

  ./a.out

```
Mac-mini-2:source code minhhuongnguyen$ ./a.out
Hello World
```

Trouble Shooting

- Can not find gcc:
  - Not installed yet
  - Incorrect path to gcc
  - …
- Permission denied
  - Change the permission to access a file ($chmod +x … in Linux or MacOs)
- ...

Basic Coding Conventions in C

Why Coding Convention?

- Enhance
  - Readability
  - Maintainability
  - Reusability
  - Efficiency
  - Coding productivity
- Reduce
  - Risk, bugs
  - Cost

Formating Code

- Maximum length of source lines: 78
- Braces:
  - Open-brace of a function, struct, enum is at column #1
  - Contents inside braces are indented (by clicking TAB)
  - Always put braces around if-else
- NOT put parenthesis () next to keywords, function names
- Use an empty line to separate blocks if necessary

```
9   void foo ()
10  {/* Open brace in column #1 here */
11      if (condition){
12          //lines of block...
13      }
14      else{
15          //lines of block...
16      }
17
18      while (condition)
19      {
20          //lines of block...
21      }
22
23      do{
24          //lines of block...
25      }while (condition)
26  }
27
28  struct foo
29  {/* Open brace in column #1 here */
30    int a, b;
31  }
```

Using spaces to indent be like



➔ Always use TAB to indent your code

Formating Code (cont)

- Use Header File Guards #define
  - Goal: to prevent multiple inclusion
  - Format: #define <PROJECT>_<PATH>_<FILE>
  - Eg:

```
#ifndef FOO_BAR_BAZ
#define FOO_BAR_BAZ

...

#endif  // FOO_BAR_BAZ
```

Naming

- Variables & functions:
  - All lower-case letters & _ (as separator)

```
int ignore_space_change_flag;
void foo() {}
```

  - Global variables should be prepended with a 'g_'
  - Pointers: prefix p, use * next to the datatype

```
int* pName;
```

- Constants, #define: all uppercase letter & _ (as separator)

```
const double PI = 3.14;
#defind MY_CONSTANT 123
```

- Enum

```
enum PinStateType {
    PIN_OFF,
    PIN_ON
};
```